WAMO Labs — Round 2 Interview Cheatsheet

Muhammad Faizan Yousaf | Sunday April 26, 2026 | 6:00 PM Onsite

SECTION 1 — Academic Intro Questions

Q: Tell us about your academic background and subjects.

➜ I'm in my final semester at FAST-NUCES, graduating June 2026. My core subjects cover the full CS stack — DSA, OS, DBMS, Networks, Software Engineering, and Algorithms. On the AI side I took Artificial Intelligence, NLP, Deep Learning for Perception, and Generative AI as electives. So I have both strong fundamentals and applied AI depth.

Q: What electives did you choose and why?

➜ In Semester 6 I chose Web Programming over SMD — I wanted practical full-stack skills I could apply immediately in projects. In Semester 7 I chose NLP over Computer Vision because my interest is in language models and text-based AI systems. In Semester 8 I chose Software Testing over Cloud Computing — I felt testing is something I could learn deeply through the course, while cloud is something I can self-learn practically. That decision was about maximizing what formal study adds.

■ *They may push back. Stand firm — this shows reasoning, not avoidance.*

Q: What is your GPA and how do you feel about it?

➜ My CGPA is 3.30/4.0. It's solid — I'm not top of class, but it reflects consistent performance across 130 credit hours covering hard subjects like DSA, Algorithms, Theory of Automata, and OS. My stronger semesters were when I was also building real projects alongside coursework.

■ *Don't apologize for it. Contextualize it.*

Q: What feedback do you have from Round 1?

➜ Round 1 was strong overall — I was comfortable with OOP, JavaScript concepts, and DB schema design. If anything, I could have been faster on the DSA problem — I got the solution but took time to think through the edge cases. I've reviewed those areas since.

■ **Say this exactly — it shows self-awareness without underselling yourself.**

Q: What are your salary expectations?

➜ I'm looking for somewhere in the 80,000–120,000 PKR range based on the role and my skill set. I'm flexible if the learning opportunity and growth path are strong.

■ *WAMO posted 70k–150k. Don't anchor too low — you have real projects.*

Q: What do you expect from this role?

➜ I want to work on real production problems, get code reviewed by engineers who are better than me, and grow fast. I'm not looking for a slow ramp-up — I want to contribute from week one.

SECTION 2 — Subject Knowledge They May Probe

Data Structures (C grade — be ready)

➜ Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, Hashing. Know time complexities cold.

Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)	O(1)
Stack/Queue	O(n)	O(n)	O(1)	O(1)
Hash Map	O(1)	O(1)	O(1)	O(1)
Binary Tree	O(n)	O(n)	O(n)	O(n)
BST (avg)	O(log n)	O(log n)	O(log n)	O(log n)

OOP (your strong area — keep it sharp)

Pillar	One Line	JS Example
Encapsulation	Bundle data + hide internals	Private #field with getter/setter
Abstraction	Hide complexity, show interface	Abstract class / interface pattern
Inheritance	Child reuses parent behavior	class Dog extends Animal
Polymorphism	Same call, different behavior	speak() differs per subclass

OS Concepts (they may ask — you got B in OS)

➜ Process vs Thread: A process is an independent program with its own memory. A thread is a lighter unit inside a process, sharing memory.

➜ Deadlock: 4 conditions — Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait. All four must hold simultaneously.

➜ Scheduling: FCFS, SJF, Round Robin, Priority. Round Robin is most common in real systems — each process gets a fixed time slice.

➜ Memory Management: Paging splits memory into fixed-size pages. Segmentation uses variable-size segments. Virtual memory lets processes use more memory than physically available.

DBMS (expect questions — you got B in DB)

➜ Normalization: 1NF = atomic values. 2NF = no partial dependency. 3NF = no transitive dependency. BCNF = every determinant is a candidate key.

➜ ACID: Atomicity (all or nothing), Consistency (valid state), Isolation (no interference), Durability (persists after crash).

➜ Indexing: B-Tree index speeds up reads on WHERE/JOIN columns. Slows writes. Use on high-cardinality, frequently-queried columns.

Algorithms (B in DAA — know these)

➜ Sorting: Merge Sort O(n log n). Quick Sort O(n log n) avg, $O(n^2)$ worst. Bubble Sort $O(n^2)$ — never use in production.

➜ Graph: BFS uses a queue (level-by-level). DFS uses a stack/recursion (depth-first). Dijkstra for shortest path (weighted graph).

➜ Dynamic Programming: Break problem into overlapping subproblems, store results. Examples: Fibonacci, Knapsack, LCS.

NLP (your elective — they may go deep)

➜ Tokenization → Embeddings → Attention → Transformer. BERT is encoder-only (classification/NER). GPT is decoder-only (generation). T5 is encoder-decoder (translation/summarization).

➜ RAG: Retrieve relevant documents using vector similarity, then pass as context to an LLM to generate grounded answers.

➜ BLEU score: measures n-gram overlap between generated and reference text. Higher = better. Your NMT achieved BLEU 0.45+ — that's solid for sequence-to-sequence.

SECTION 3 — JavaScript Conceptual Questions

Q: var vs let vs const?

➜ var is function-scoped, hoisted and initialized to undefined. let and const are block-scoped, hoisted but NOT initialized (TDZ). const can't be reassigned. Use const by default, let when you need to reassign, avoid var.

```
console.log(a); // undefined — var hoisted
var a = 5;
console.log(b); // ReferenceError — TDZ
let b = 10;
```

Q: What is the Temporal Dead Zone?

➜ The period between when a let/const variable is hoisted and when it's initialized. Accessing it in this window throws ReferenceError — the variable exists but is in a 'dead' state.

■ *TDZ = the gap between top of block scope and the actual declaration line.*

Q: What is hoisting?

➜ JS moves declarations to the top of their scope before execution. var declarations are hoisted AND initialized to undefined. Function declarations are fully hoisted. let/const are hoisted but NOT initialized.

Q: What is a closure?

➜ A function that retains access to variables from its outer scope even after that outer function has finished executing.

```
function counter() {
let count = 0;
return () => ++count; // closes over 'count'
}
const inc = counter();
inc(); // 1 | inc(); // 2 — count persists
```

Q: Arrow function vs regular function?

➜ Arrow functions don't have their own 'this' — they inherit it from the surrounding scope. Regular functions have their own 'this' bound at call time. Arrow functions cannot be used as constructors.

```
const obj = {
name: 'Faizan',
regular: function() { return this.name; }, // 'Faizan'
arrow: () => this.name, // undefined — 'this' is outer scope
};
```

Q: What is the event loop?

➜ JS is single-threaded. The event loop monitors the call stack and callback queues. When the call stack is empty, it picks tasks from the microtask queue (Promises) first, then macrotask queue (setTimeout, setInterval).

■ *Order: Call Stack → Microtask Queue (Promises, .then) → Macrotask Queue (setTimeout)*

Q: Promise.all vs Promise.race vs Promise.allSettled?

➜ Promise.all: waits for ALL — fails fast if any rejects. Promise.race: first to settle wins (resolve or reject). Promise.allSettled: waits for ALL regardless of outcome — gives result/reason for each.

Q: == vs ===?

➜ == coerces types before comparing. === checks value AND type — no coercion. Always use === in production.

```
0 == '0' // true
0 === '0' // false
```

Q: What is 'this' in JavaScript?

➜ Refers to the object that is currently executing the function. In a method: the object. In a regular function: global (or undefined in strict mode). In an arrow function: the enclosing scope's this. In a constructor: the new object.

Q: map vs filter vs reduce?

➜ map: transforms each element, returns same-length array. filter: keeps elements that pass a test, returns shorter array. reduce: accumulates elements into a single value.

```
[1,2,3].map(x => x*2); // [2,4,6]
[1,2,3].filter(x => x > 1); // [2,3]
[1,2,3].reduce((a,b) => a+b); // 6
```

Q: What is null vs undefined vs NaN?

➜ undefined: variable declared but not assigned. null: intentional absence of value (assigned explicitly). NaN: result of invalid math operation — type is 'number' but value is not a number.

```
typeof undefined // 'undefined'
typeof null // 'object' (JS quirk)
typeof NaN // 'number' isNaN(NaN) // true
```

Q: Shallow copy vs deep copy?

➜ Shallow copy copies references — nested objects are still shared. Deep copy fully duplicates all levels — no shared references.

```
// Shallow
const b = { ...a }; // or Object.assign({}, a)
// Deep
const c = JSON.parse(JSON.stringify(a)); // simple deep copy
// or use structuredClone(a) in modern JS
```

SECTION 4 — DSA: Two Sum + 2nd Highest + Others

Q: Two Sum — given array and target, return indices of two numbers that add up to target.

➜ Use a hash map. Store each number and its index. For each element, check if (target - element) already exists in the map. O(n) time — say this out loud.

```
function twoSum(nums, target) {
const map = {};
for (let i = 0; i < nums.length; i++) {
const diff = target - nums[i];
if (map[diff] !== undefined) return [map[diff], i];
map[nums[i]] = i;
}
}
twoSum([2,7,11,15], 9); // [0,1]
```

■ *Always state: O(n) time, O(n) space. Better than sorting which is O(n log n).*

Q: Find 2nd highest number in an array.

➜ Single pass with two variables. No sorting needed — O(n) time, O(1) space.

```
function secondHighest(arr) {
let first = -Infinity, second = -Infinity;
for (let n of arr) {
if (n > first) { second = first; first = n; }
else if (n > second && n !== first) second = n;
}
return second === -Infinity ? null : second;
}
secondHighest([3,1,4,1,5,9,2,6]); // 6
secondHighest([5,5,5]); // null — no distinct 2nd
```

■ *Handle duplicates and edge case where all elements are same — they'll test this.*

Q: Reverse a string / array.

➜ Two-pointer approach — swap from both ends moving inward. O(n) time, O(1) space.

```
function reverse(arr) {
let l = 0, r = arr.length - 1;
while (l < r) {
[arr[l], arr[r]] = [arr[r], arr[l]];
l++; r--;
}
return arr;
}
```

Q: Check if string is a palindrome.

➜ Clean the string, compare from both ends. O(n) time.

```
function isPalindrome(s) {
s = s.toLowerCase().replace(/[^a-z0-9]/g, '');
let l = 0, r = s.length - 1;
while (l < r) {
if (s[l] !== s[r]) return false;
l++; r--;
}
return true;
}
```

Q: Valid Parenthesis (Stack).

➜ Push opening brackets. For each closing bracket, pop and check match. Empty stack at end = valid.

```
function isValid(s) {
const stack = [], map = {')':'(',']':'[','}':'{'};
for (let c of s) {
if ('([{'.includes(c)) stack.push(c);
else if (stack.pop() !== map[c]) return false;
}
return stack.length === 0;
}
```

SECTION 5 — ERD, Multiplicity & DBMS

ERD Core Concepts

Term	Meaning	Example
Entity	A real-world object/thing	User, Order, Product
Attribute	Property of an entity	user_id, name, email
Primary Key	Unique identifier for each row	user_id PK
Foreign Key	Links to PK of another table	order.user_id → users.user_id
Relationship	Association between entities	User PLACES Order

Multiplicity — How to resolve M:N

Type	Meaning	Resolution	Example
1:1	One to One	FK in either table	Person — Passport
1:M	One to Many	FK on the Many side	User — Orders (order.user_id FK)
M:N	Many to Many	Junction/Bridge table with both FKs	Students — Courses → Enrollment(student_id, course_id)

InDrive-type ERD — Draw This Cold

```
Rider(rider_id PK, name, phone, rating)
Driver(driver_id PK, name, phone, vehicle, rating, is_available)
Booking(booking_id PK, rider_id FK → Rider, driver_id FK → Driver,
pickup, dropoff, status, price, created_at)
-- Rider 1:M Booking | Driver 1:M Booking | Rider M:N Driver via Booking
```

Sample Query

```
-- All completed rides for rider 101
SELECT b.booking_id, d.name AS driver, b.pickup, b.dropoff, b.price
FROM Booking b
JOIN Driver d ON b.driver_id = d.driver_id
WHERE b.rider_id = 101 AND b.status = 'completed'
ORDER BY b.created_at DESC;
```

Q: What is normalization and when do you use it?

➜ Removing data redundancy by splitting into related tables. Use for write-heavy systems needing data integrity (banking, e-commerce). Denormalize for read-heavy systems (analytics, dashboards) to reduce JOINs.

Q: What are ACID properties?

➜ Atomicity: all or nothing. Consistency: DB stays in valid state. Isolation: concurrent transactions don't interfere. Durability: committed data survives crashes via write-ahead logging.

SECTION 6 — Consistency Check: What They'll Re-Test from Round 1

■ Round 2 deliberately re-asks Round 1 topics in different words. If your answer changes, you fail. Lock these in:

Topic	Round 1 phrasing	Round 2 phrasing	Same answer?
Composition	Composition vs Aggregation	What happens to Part A if Object B is destroyed?	YES — Part A dies with B in composition
Hoisting	What is hoisting?	Why does console.log(x) before let x = 5 throw a ReferenceError?	YES — TDZ, not initialized
Promise.all	What does Promise.all do?	What happens if one of 5 API calls fails in Promise.all?	YES — entire batch rejects
Normalization	What is normalization?	Would you normalize a table for a reporting dashboard?	NO — denormalize for read-heavy
Polymorphism	Explain polymorphism	Can two different classes have a method with the same name?	YES — that is polymorphism

Think out loud. State your approach before coding. Admit uncertainty — never guess silently.